DhanwantariAI — End-to-End Architecture Document

Version: 3.0
Date: 2025-07-17
Product: DhanwantariAI — Offline-First Clinical Decision Support System
Entity: AppScale LLP (LLPIN: ACP-6024)
Author: Satyam Kumar Das
Regulatory Classification: CDSCO Class A SaMD (Software as a Medical Device)

Table of Contents

1. System Overview
2. Device Tier Architecture
3. Mobile Application Layer
4. AI & Clinical Safety Engine
5. Retrieval Architecture
6. Confidence Scoring Engine
7. LLM Routing Engine
8. AWS Cloud Infrastructure
9. Three-Agent Bedrock Layer
10. Training Pipeline
11. Sync Architecture
12. Security & Privacy
13. Liability & Medical Governance
14. Data Sources & Knowledge Base
15. Multilingual Architecture
16. Observability & Telemetry
17. Project File Structure
18. Implementation Status

---

1. System Overview

DhanwantariAI is an **offline-first Clinical Decision Support System (CDSS)** designed for ASHA workers and ANMs in rural India. The system operates on a 4-tier LLM architecture where the lowest-capability devices run purely on deterministic rule engines, while higher-tier devices optionally download on-device LLMs (Gemma 3 1B/4B), with AWS Bedrock (Claude 3 Haiku) as an online escalation path.

Core Design Principles

- **Offline-First:** All clinical decision support works without connectivity
- **Privacy-by-Design:** DPDP Act 2023 compliant; no PII leaves the device
- **Safety-First:** 18 red-flag rules fire BEFORE any LLM/retrieval call
- **Tiered Capability:** Adapts AI stack to device hardware (RAM/CPU/disk)
- **Cost Constrained:** Target ₹120/month cloud budget at 1,000 escalated queries/day

High-Level Data Flow

```
User Input (symptoms/chat)
    │
    ▼
┌──────────────────────────────────────────┐
│  ClinicalSafetyEngine (18 RedFlagRules)  │  ← ALWAYS FIRST
│  RF001–RF018 with ICD-10 codes           │
└──────────────┬───────────────────────────┘
               │
               ▼
┌──────────────────────────────────────────┐
│  DiagnosisEngine (orchestrator)          │
│  ├── symptomMatcher (deterministic)      │
│  ├── RuleEngine (risk classification)    │
│  └── HybridRetrieval (FTS5+PI+Vector)    │
└──────────────┬───────────────────────────┘
               │
               ▼
┌──────────────────────────────────────────┐
│  ConfidenceScorer (6-signal weighted)    │
│  └─ score < 0.55 → escalate to Bedrock   │
└──────────────┬───────────────────────────┘
               │
               ▼
┌──────────────────────────────────────────┐
│  LLMEngine (4-tier routing)              │
│  T1: On-device GGUF (llama.rn)           │
│  T2: Network llama.cpp server            │
│  T3: AWS Bedrock (Claude Haiku)          │
│  T4: Offline pattern fallback            │
└──────────────┬───────────────────────────┘
               │
               ▼
┌──────────────────────────────────────────┐
│  Liability Layer                         │
│  ├── SourceCitationEngine                │
│  ├── DisclaimerManager                   │
│  └── AuditLogger (no PII)                │
└──────────────────────────────────────────┘
```

---

2. Device Tier Architecture

File: `src/ai/DeviceCapabilityDetector.ts`

5-Signal Detection (v2.2 §4.2)

Signal	Source	Purpose
`totalRAM`	`DeviceInfo.getTotalMemory()`	Primary tier boundary
`isLowRam`	`DeviceInfo.isLowRamDevice()`	Android system low-RAM flag
`freeDisk`	`DeviceInfo.getFreeDiskStorage()`	Model download feasibility
`cpuArch`	`DeviceInfo.supportedAbis()`	ARM64 required for LLM
`apiLevel`	`DeviceInfo.getApiLevel()`	NNAPI ≥ 27, min API 26

Tier Definitions

Tier	RAM	AI Stack	LLM Model
TIER_1	< 2 GB, or !ARM64, or API < 26, or isLowRam	PageIndex + FTS5 + Rule Engine only	None
TIER_2	2–8 GB, ARM64, API ≥ 26	Full retrieval + optional on-device LLM	Gemma 3 1B int4 (529 MB)
TIER_3	≥ 8 GB, ARM64, API ≥ 26	Full retrieval + on-device LLM	Gemma 3 4B int4 (~2.5 GB)

Key Implementation Details

- Tier is detected at first launch and **cached permanently** in AsyncStorage (`dhanwantari_device_profile_v2`)
- `nnApiSupported = apiLevel >= 27 && cpuArch === 'arm64'`
- Model download is user-initiated (opt-in), WiFi required for 4B model
- `select_model_variant()` in `client_sync_engine.py` selects the highest-fitting variant per `model_variants.json`

---

3. Mobile Application Layer

Platform: React Native (TypeScript)
State Management: Redux Toolkit + Redux Persist (AsyncStorage)
Database: op-sqlite (SQLite with sqlite-vec extension)

State Architecture

File: `src/store/store.ts`

```
rootReducer
  ├── profileReducer  → profiles[], activeProfileId
  ├── chatReducer     → sessions{profileId: ChatMessage[]}, isLLMConnected
  ├── symptomReducer  → selectedSymptoms[], lastAnalysis, analysisHistory
  ├── deviceReducer   → profile, consentGranted, llmDownloaded,
llmDownloadProgress
  └── llmApi.reducer  → RTK Query cache (not persisted)
```

Persisted slices: `profile`, `chat`, `symptom`, `device` (whitelist in redux-persist config).

Database Schema (db.ts)

Table	Purpose
`profiles`	User profiles (CRUD)
`diseases`	146 disease records from symptom_disease_mapping.json
`disease_symptoms`	Normalised symptom→disease reverse index
`reports`	Indexed health report documents
`report_chunks`	Individual tree-node chunks per report
`chunks_fts`	FTS5 virtual table over chunk text
`disease_vectors`	sqlite-vec virtual table, 384-dim MiniLM-L6-v2 embeddings

Feature Screens

```
src/features/
  ├── chat/            — Chat screen with LLM integration
  ├── classifications/ — Disease classification results
  ├── consent/         — DPDP consent collection
  ├── medicine/        — JanAushadhi medicine search
  ├── profile/         — User profile management
  ├── referral/        — Referral guidance display
  └── symptoms/        — Symptom checker UI
```

Services Layer

File	Purpose
`db.ts`	op-sqlite initialization, schema creation, profile CRUD
`llmApi.ts`	RTK Query API for local llama.cpp server (health check, chat completion)
`offlineFallback.ts`	Pattern-matched responses for complete offline mode
`reportIndexer.ts`	Offline document indexing via react-native-pageindex
`syncEngine.ts`	TypeScript sync policy engine (mirrors Python client_sync_engine.py)

---

4. AI & Clinical Safety Engine

4.1 ClinicalSafetyEngine (MUST BE FIRST)

File: `src/ai/ClinicalSafetyEngine.ts`
Architecture ref: v2.2 §6.1–6.2

Implements **18 typed RedFlagRule structs (RF001–RF018)** that run BEFORE any retrieval or LLM call.

Rule ID	Symptom	Referral	ICD-10	Source
RF001	Loss of consciousness	HOSPITAL	R55	WHO IMCI 2014 §3.1
RF002	Seizure/convulsion	HOSPITAL	G40	IAP PALS 2018 §2
RF003	Eclampsia	HOSPITAL	O15	RCOG Green-top 10A 2022
RF004	Severe chest pain	HOSPITAL	I21	ESC STEMI 2023
RF005	Respiratory failure/cyanosis	HOSPITAL	J96	WHO IMCI 2014 §3.2
RF006	Stroke symptoms	HOSPITAL	I63	AHA/ASA Stroke 2023
RF007	Postpartum haemorrhage	HOSPITAL	O72	WHO PPH 2012; FOGSI 2019
RF008	Anaphylaxis	HOSPITAL	T78.2	WAO Anaphylaxis 2020
RF009	Septic shock	HOSPITAL	A41	SSC International 2021
RF010	Suspected meningitis	HOSPITAL	G03	WHO Bacterial Meningitis 2018
RF011	Severe/cerebral malaria	HOSPITAL	B50.0	NVBDCP 2021 §4.3
RF012	Haemoptysis/haematemesis	HOSPITAL	R04.2	RNTCP TB 2019
RF013	Suspected poisoning	HOSPITAL	T65.9	AIIMS Poisoning 2020
RF014	Snake bite	HOSPITAL	T63.0	WHO Snakebite 2016
RF015	Neonatal convulsion	HOSPITAL	P90	NNF IMNCI 2014 §5.2
RF016	Dengue shock/haemorrhagic	FRU	A91	NVBDCP Dengue 2015 §3.4
RF017	Obstructed/prolonged labour	FRU	O65	FOGSI MNHRC 2019
RF018	Drowning/near-drowning	HOSPITAL	T75.1	ILCOR Resuscitation 2020 §8

Key behavior: On CRITICAL evaluation, `suppressLLM = true` — blocks all LLM calls to prevent contradictory guidance.

4.2 RuleEngine

File: `src/ai/RuleEngine.ts`

Deterministic clinical decision support layer. Runs BEFORE any LLM/vector retrieval.

- **Risk levels:** `IMMEDIATE` → `URGENT` → `ROUTINE`
- **Referral levels:** `ASHA_MANAGE` → `PHC` → `CHC` → `FRU` → `HOSPITAL`
- Category-based referral overrides (e.g., Neurological → CHC, Pregnancy → PHC)
- Immediate actions lookup per risk level (e.g., "Call 108", "Keep patient lying down")

4.3 DiagnosisEngine (Orchestrator)

File: `src/ai/DiagnosisEngine.ts`

Central orchestrator combining all analysis steps in order:

```
Step 0: ClinicalSafetyEngine.evaluateSafety() — MUST BE FIRST
Step 1: symptomMatcher.analyzeSymptoms() — deterministic scoring
Step 2: RuleEngine.assessRisk() — red flag / referral classification
Step 3: HybridRetrieval.retrieveHybrid() — async, graceful offline
Step 4: Combined confidence score (matcher 0.5 + retrieval 0.4 + severity
0.1)
Step 5: Escalation decision (suppressLLM / IMMEDIATE / low confidence /
URGENT+medium)
Step 6: Personalised summary builder
```

Escalation logic: - `suppressLLM = true` → NO escalation (safety override) - `riskLevel === 'IMMEDIATE'` → escalate for Bedrock confirmation - `confidenceScore < CONFIDENCE_AUTO_ESCALATE (0.55)` → escalate - `confidenceScore < CONFIDENCE_MEDIUM (0.70) && riskLevel === 'URGENT'` → escalate

---

5. Retrieval Architecture

5.1 HybridRetrieval (Orchestrator)

File: `src/retrieval/HybridRetrieval.ts`

Runs all three retrieval methods **in parallel** with graceful fallback:

```
Promise.all([
  1. FTS5Search.searchBySymptomList()    — SQLite FTS5 (offline)
  2. PageIndexNavigator.searchPageIndex() — bundled JSON trees (offline)
  3. VectorSearch.searchDiseasesByVector() — Qdrant Cloud (online, fallback
[])
])
    |
    ▼
ResultReconciler.reconcile()  → agreements[], conflicts[]
ConfidenceScorer.scoreConfidence() → score, escalationReason
    |
    ▼
RetrievalBundle { pageIndexNodes, ftsResults, vectorMatches, agreements,
conflicts, confidenceScore }
```

5.2 FTS5Search

File: `src/retrieval/FTS5Search.ts`

- Uses op-sqlite FTS5 virtual table (`chunks_fts`)
- BM25 ranking via `ORDER BY rank`
- `searchDiseasesFTS(query)` — keyword search over disease_name, symptom_text
- `searchReportsFTS(query)` — search indexed report chunks
- `searchBySymptomList(symptoms)` — wraps symptoms in quotes for phrase matching, joined by OR
- Query sanitization: strips `'"\\` characters

5.3 PageIndexNavigator

File: `src/retrieval/PageIndexNavigator.ts`

- Traverses **bundled PageIndex JSON trees** (no file I/O — loaded via `require()`)
- Sources: `clinical_protocols_pageindex.json`, `referral_guidelines_pageindex.json`
- BM25-inspired word-overlap scoring between query tokens and node content
- Nodes are pre-flattened on first call (memoized)
- Returns top-K `PageIndexNode[]` sorted by overlap score

5.4 VectorSearch

File: `src/retrieval/VectorSearch.ts`

- **Qdrant Cloud** REST API via axios
- 3 collections: `disease_symptoms`, `medicines`, `clinical_protocols`
- 384-dimensional MiniLM-L6-v2 embeddings
- Timeout: 8000ms, graceful fallback to `[]` on network failure
- API key auth via `Config.QDRANT_API_KEY`

5.5 ResultReconciler

File: `src/confidence/ResultReconciler.ts`

- **Agreements:** disease names appearing in ≥2 of 3 sources (FTS ∩ Vector, FTS ∩ PageIndex, Vector ∩ PageIndex)
- **Conflicts:** FTS top result ≠ Vector top result
- Used for confidence scoring and transparency

---

6. Confidence Scoring Engine

File: `src/confidence/ConfidenceScorer.ts`
Architecture ref: v2.2 §8.1

6-Signal Weighted Scoring

Signal	Weight	Description
`retrievalCoverage`	0.20	Fraction of 3 sources with results (FTS/PI/Vec)
`sourceAgreement`	0.25	Count of disease names in ≥2 sources (0.083 per agreement, max 3)
`diseaseScoreStrength`	0.20	FTS count quality (0.06–0.10) + Vector top score (0–0.10)
`symptomSpecificity`	0.15	Number of symptoms reported (1→0.04, 2→0.08, 3-4→0.12, 5+→0.15)
`patientContextMatch`	0.10	Gender alignment (match→0.10, both→0.10, mismatch→0.02, unknown→0.05)
`knowledgeDepth`	0.10	PageIndex node count (1→0.04, 2→0.07, 3+→0.10)

Total maximum: 1.00

Confidence Thresholds (from config.ts)

Threshold	Value	Action
`CONFIDENCE_HIGH`	0.85	High confidence — local result used
`CONFIDENCE_MEDIUM`	0.70	Medium — Bedrock recommended if risk is URGENT
`CONFIDENCE_AUTO_ESCALATE`	0.55	Auto-escalate to Bedrock
`CONFIDENCE_VERY_LOW`	0.40	Very low — offline fallback + referral

---

7. LLM Routing Engine

File: `src/cloud/LLMEngine.ts`

4-Tier Priority (Highest → Lowest)

Tier	Source	Condition	Timeout
1. On-device GGUF	llama.rn (llama.cpp)	TIER_2/3 + model downloaded	—
2. Network llama.cpp	`Config.LLM_BASE_URL/health`	Health check passes (2s)	30s
3. AWS Bedrock	`Config.BEDROCK_PROXY_URL`	URL configured + TLS pinned	20s
4. Offline fallback	`offlineFallback.ts`	Always available	—

Shared System Prompt

```
You are DhanwantariAI, a clinical decision support assistant for Indian ASHA
workers.
You have knowledge of 146 common diseases, JanAushadhi medicines, Ayurvedic
remedies,
and India-specific health protocols. [Patient context]. Respond concisely and
always
recommend professional consultation for serious symptoms.
```

Offline Fallback Patterns

Quick-response patterns for common queries when all LLM tiers are unavailable: - Dengue danger signs - Fever in pregnancy - TB DOTS medicines with JanAushadhi prices - Diabetes JanAushadhi medicines - Symptom check results (parsed from structured SymptomAnalysisScreen output) - Greeting, BMI, medicine query, symptom checker prompt

---

8. AWS Cloud Infrastructure

Region: `ap-south-1` (Mumbai) — DPDP Act 2023 data residency
Account: `034250960622`

8.1 Provisioned Resources (setup_aws.py)

Resource	ARN / URL	Configuration
Lambda	`arn:aws:lambda:ap-south-1:034250960622:function:DhanwantariBedrockProxy`	Python 3.12, ARM64, 256MB, 29s timeout
IAM Role	`arn:aws:iam::034250960622:role/DhanwantariBedrockLambdaRole`	Least-privilege: Bedrock InvokeModel, DynamoDB, Logs, Guardrails, CloudWatch metrics, SNS
API Gateway	`https://4fx87rqhze.execute-api.ap-south-1.amazonaws.com/prod/escalate`	HTTP API, POST / escalate, 50 RPS throttle, prod stage
DynamoDB	`arn:aws:dynamodb:ap-south-1:034250960622:table/bedrock_usage`	PAY_PER_REQUEST, TTL enabled, pk=date_key, sk=metric
Bedrock Guardrail	ID: `4kvilnflnw3o`	Topic deny, word filters, PII anonymization
SNS Topic	`arn:aws:sns:ap-south-1:034250960622:DhanwantariAlerts`	Alert notifications
Secrets Manager	`arn:aws:secretsmanager:ap-south-1:034250960622:secret:DhanwantariAI/QdrantApiKey-zHAgmc`	Qdrant API key storage
Bedrock Model	`anthropic.claude-3-haiku-20240307-v1:0`	Claude 3 Haiku — ~₹0.004/query

8.2 CloudWatch Alarms

Alarm	Threshold	Period
HighEscalationRate	> 40% queries escalated	1 hour
BedrockCostSpike	> $3	7 days
BedrockErrors	> 10 errors	1 hour

8.3 Lambda Handler (index.py)

```
POST /escalate
  ├── Input validation (modelId, messages, system)
  ├── Server-side PII strip (Aadhaar, phone, PIN, name, village)
  ├── Cost controller
  │   ├── Daily cap: 10,000 queries (DynamoDB atomic increment)
  │   └── Monthly budget: $15
  ├── Invoke Bedrock (Claude 3 Haiku)
  ├── Emit CloudWatch metrics
  │   ├── EscalationRate (count)
  │   ├── BedrockCost (USD)
  │   ├── BedrockLatencyMs (latency)
  │   └── BedrockInvokeError (error count)
  └── Structured audit log (no PII)
```

8.4 Qdrant Cloud

- **URL:** `https://46696dcb-3c39-41db-bab3-80d80027e898.eu-west-2-0.aws.cloud.qdrant.io:6333`
- **Region:** eu-west-2 (Free Tier)
- **Embedding model:** MiniLM-L6-v2 (384 dimensions)
- **Collections:**
- `disease_symptoms` — 384-dim, disease_id + disease_name + symptom_text
- `medicines` — 384-dim, medicine_name + category + disease_ids + description
- `clinical_protocols` — 384-dim, protocol_name + content + referral_level

---

9. Three-Agent Bedrock Layer

Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    Mobile App                            │
│  ├── online + low confidence → Agent 2 (Inference)       │
│  └── deep question button  → Agent 3 (Knowledge)         │
└──────────────┬──────────────────────────┬────────────────┘
               │                          │
               ▼                          ▼
┌──────────────────────┐  ┌──────────────────────────┐
│  Agent 2: Inference  │  │  Agent 3: Knowledge Base │
│  bedrock_inference_  │  │  bedrock_kb_agent.py     │
│  agent.py            │  │  RAG over clinical data  │
│  Claude Haiku        │  │  Amazon Nova Lite        │
└──────────────────────┘  └──────────────────────────┘
                                  │
                        ┌─────────┘
                        ▼
           ┌──────────────────────┐
           │ Agent 1: Data Sync   │   ← EventBridge / scheduled
           │ bedrock_sync_        │
           │ agent.py             │
           │ Daily delta + CDN    │
           └──────────────────────┘
```

Agent 1: Data Sync Agent (`bedrock_sync_agent.py`)

Runs daily on EventBridge schedule: 1. Fetch latest Jan Aushadhi prices (PMBJP catalogue) 2. Check ICMR / NTEP / WHO-IN guideline updates 3. Generate RFC 6902-style disease-level delta (replaced/added/removed) 4. Upload delta + full package (gzipped) to S3 5. Publish `sync_manifest.json` to CDN origin (`https://cdn.dhanwantariai.in`) 6. Multi-variant model updates (gemma3-1b + gemma3-4b in same manifest)

Agent 2: Inference Agent (`bedrock_inference_agent.py`)

- **Purpose:** LLM inference for Tier 1 devices (< 2 GB RAM)
- **Model:** `anthropic.claude-haiku-4-5`
- **API:** `POST /inference`
- **Input:** Pre-scored disease results from on-device engine (top_diseases, symptoms, profile_flags)
- **Output:** Analysis paragraph + follow-up questions + disclaimer
- **Rate limit:** 20 calls/day per device_id (SHA-256 hash)
- **Privacy:** device_id is always SHA-256 hash; no symptom history stored server-side
- **Deploy:** Lambda Python 3.12, ARM64 Graviton, 512 MB, 30s timeout

Agent 3: Knowledge Base Agent (`bedrock_kb_agent.py`)

- **Purpose:** Deep clinical questions via RAG (drug interactions, guidelines, etc.)
- **Model:** `amazon.nova-lite-v1:0` (fast + cheap for RAG)
- **API:** `POST /knowledge`
- **Source:** Bedrock Knowledge Base (S3 + OpenSearch Serverless vector store)
- **KB Sources:** disease_db_full.json, ICMR guidelines PDFs, JanAushadhi catalogue, drug_interactions.json
- **Input:** question + optional context (disease_ids, medicine_names)
- **Output:** Answer + source citations + disclaimer
- **Max question length:** 500 chars (prompt injection protection)

---

10. Training Pipeline

File: `training/train.py`
Config: `sync/model_variants.json`

Training Configuration (from model_variants.json)

Parameter	Value
Method	QLoRA (4-bit quantized LoRA) via Unsloth
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.1
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Learning rate	1e-4 (cosine scheduler)
Epochs	5
Batch size	4 (gradient accumulation 4 → effective 16)
Max sequence length	2048
Warmup ratio	0.1
Weight decay	0.05
Early stopping patience	3
bf16	true

Model Variants

Variant	Base Model	Parameters	Size (GGUF Q4_K_M)	Min RAM	Hardware
`gemma3-1b-dhanwantari-ft`	google/gemma-3-1b-it	1B	~529 MB	1.5 GB	RTX 3090 24GB (~45 min)
`gemma3-4b-dhanwantari-ft`	google/gemma-3-4b-it	4B	~2.5 GB	3 GB	A100 80GB (~2.5 hr)

Language Pack Architecture

- **Base model** trained on English data → LoRA adapter (~529 MB for 1B)
- **Language packs** = LoRA adapter (~40 MB) + UI translation sidecar (~3 MB)
- User downloads base model **once**; only the adapter changes per language
- Training data in `training_data/lang/<code>/train.jsonl`
- System prompts in `training/lang/<code>/system_prompt.txt`
- Planned languages: Hindi (hi), Odia (or), Marathi, Kannada, Tamil, Telugu

Export Formats

Format	Use Case	File
LiteRT .bin	On-device GPU delegate (int4)	`gemma3-Xb-dhanwantari-ft.bin`
GGUF Q4_K_M	llama.cpp CPU fallback	`gemma3-Xb-dhanwantari-ft.Q4_K_M.gguf`

Training Data

File	Purpose
`training_data/dhanwantari_train.jsonl`	English training data (ShareGPT format)
`training_data/dhanwantari_val.jsonl`	English validation data
`training_data/lang/hi/`	Hindi training data

Prompt Format (ShareGPT)

```
{"conversations": [
  {"from": "system", "value": "..."},
  {"from": "human", "value": "..."},
  {"from": "gpt", "value": "..."}
]}
```

---

11. Sync Architecture

11.1 Client-Side Sync Policy Engine

Files: `sync/client_sync_engine.py` (Python reference), `src/services/syncEngine.ts` (TypeScript port)

Sync Decision Matrix

Offline Duration	disease_db	price_catalog	model_weights
0–7 days	Delta (if available)	Delta	notify_only
8–30 days	Delta or Full	Full	notify_only
31–90 days	Full	Full	notify_only
> 90 days	Full + banner	Full + banner	prominent_banner

Throttle

- **Sync check interval:** 6 hours minimum between manifest checks
- **CDN manifest URL:** `https://cdn.dhanwantariai.in/sync_manifest.json`

Model Variant Selection

```
def select_model_variant(device_ram_mb, is_on_wifi):
    # Walk variants highest→lowest capability
    # Return first where min_ram_mb <= device_ram_mb AND wifi satisfied
    # Below 1500 MB → None (rule-engine only)
```

Session Record Immutability

```
SessionRecord (IMMUTABLE after creation):
  - disease_db_version    ← frozen at session close
  - price_catalog_version ← frozen at session close
  - model_weights_version ← frozen at session close

Rule: sync MUST NOT modify any SessionRecord field.
Historical prices stay as they were at diagnosis time.
```

11.2 Manifest Structure

Published by Agent 1 (Data Sync Agent) to CDN:

```
{
  "version": "20260306",
  "packages": {
    "disease_db": {
      "version": "20260306",
      "sha256": "...",
      "size_bytes": 1234567,
      "url": "https://cdn.dhanwantariai.in/disease_db_full.json.gz",
      "delta_url": "https://cdn.dhanwantariai.in/
delta_20260227_20260306.json.gz",
      "delta_from_version": "20260227"
    },
    "price_catalog": { ... },
    "model_weights": { ... }
  }
}
```

---

12. Security & Privacy

12.1 TLS Certificate Pinning (v2.2 §11.2)

File: `src/cloud/BedrockEscalationHandler.ts`

Uses `react-native-ssl-pinning` to validate the API Gateway TLS cert chain.

Level	SPKI SHA-256	Expiry
Leaf (`*.execute-api.ap-south-1.amazonaws.com`)	`CifvBerUQV7ploNbeZW/B1JMrqpNm5r+01B0EeMHkn4=`	Aug 2026
Intermediate (Amazon RSA 2048 M03)	`vxRon/El5KuI4vx5ey1DgmsYmRY0nDd5Cg4GfJ8S+bg=`	Aug 2030
Root (Amazon Root CA 1)	`+++MBgDH5WGvL9Bcn5Be30cRcL0f5O+NyoXuWtQdX1aI=`	Dec 2037

Bundled cert files: - iOS: `ios/DhanwantariMobile/amazon_rsa_2048_m03.cer` - Android: `android/app/src/main/assets/amazon_rsa_2048_m03.crt`

12.2 PII Stripping (v2.2 §11.3)

File: `src/cloud/PIIStripper.ts`

6 regex patterns strip PII before any cloud API call:

Pattern	Replacement
Aadhaar (12-digit, space/dash separated)	`[AADHAAR]`
Indian mobile (10-digit, +91/0 prefix)	`[PHONE]`
PIN codes (6-digit with "pin"/"pincode")	`[PINCODE]`
"patient " / "my patient "	`patient [NAME]`
"for " (capitalised proper noun)	`for [NAME]`
Village/gram/tehsil/taluka locations	`[LOCATION]`

Dual-layer PII protection: 1. **Client-side:** `PIIStripper.ts` strips before network call 2. **Server-side:** Lambda `index.py` re-strips (defense in depth)

12.3 Consent Management (v2.2 §12.2)

File: `src/privacy/ConsentManager.ts`

Consent Type	Requirement	Purpose
`advisory_acknowledgement`	**Mandatory**	Acknowledges DhanwantariAI is advisory-only
`cloud_escalation`	**Optional**	Permits anonymised queries to Bedrock

- Versioned consent (`CONSENT_VERSION = '1.0'`) — version bump invalidates existing consents
- Stored in AsyncStorage only — never synced to cloud

- `clearAllConsents()` for DPDP §12 data erasure requests

12.4 DPDP Act 2023 Compliance

- All data stored on-device with AES-256 encryption (op-sqlite)
- No health data retained on AWS servers (stateless Lambda)
- `device_id` always SHA-256 hash — never IMEI, phone, or PII
- AWS region ap-south-1 (Mumbai) for data residency
- Consent is free, specific, informed, and unambiguous

---

13. Liability & Medical Governance

13.1 Source Citation Engine

File: `src/liability/SourceCitationEngine.ts`
Architecture ref: v2.2 §10.1

Every clinical response includes `CitedResponse.sources[]`:

```
interface CitedResponse {
  sources: CitedSource[];     // source, url, version, pageRef
  generatedBy: GeneratedBy;  // 'pageindex' | 'llm_tier2' | 'llm_tier3' |
'bedrock'
}
```

Known source registry: WHO IMCI, WHO PPH, RNTCP, NLEM, NVBDCP, NNF IMNCI, FOGSI, AIIMS, IAP PALS, NHM.

13.2 Disclaimer Manager

File: `src/liability/DisclaimerManager.ts`
Architecture ref: v2.2 §10.2

Key	Context
`general`	All clinical output screens
`referral`	Below referral guidance
`medicine`	Medicine recommendations
`lowConfidence`	When confidence < CONFIDENCE_MEDIUM
`ayurveda`	Ayurvedic alternatives
`emergency`	IMMEDIATE risk (overlay) — "Call 108 immediately"

All disclaimers are centralized — no hardcoded strings in components.

13.3 Audit Logger (No PII)

File: `src/liability/AuditLogger.ts`
Architecture ref: v2.2 §10.3

```
interface AuditEntry {
  sessionId: string;           // generated (SID_timestamp_random)
  ts: string;                  // ISO 8601
  tier: DeviceTier;
  queryIntent: string;         // generic label — NEVER raw query text
  redFlagRulesFired: string[]; // rule IDs only (e.g. ['RF001'])
  riskLevel: RiskLevel;
  confidenceScore: number;
  escalatedTo: 'local' | 'bedrock' | null;
  sourcesCited: string[];
  kbVersion: string;
  appVersion: string;
}
```

- **Storage:** Local AsyncStorage only — never synced to cloud
- **Rotation:** Oldest entries evicted when log exceeds 500 entries
- **Strict no-PII policy:** No symptom text, disease names, or patient identifiers
- **Silent failure:** Audit errors never block clinical screens

13.4 Liability Chain (v2.2 §3)

```
Level 1: Technology Liability (AppScale LLP)
  └── System bugs, incorrect algorithm output, data pipeline errors
Level 2: Clinical Data Liability
  └── Medical accuracy of bundled content; traceable to authoritative sources
Level 3: User Responsibility (ASHA worker / ANM)
  └── Final clinical decisions rest with the health worker
```

13.5 Human Oversight (4 levels)

Level	Agent	Override
Human-in-the-loop	ASHA worker	Reviews every recommendation before acting
Supervised	PHC Medical Officer	Validates referral decisions
Monitoring	State NHM Officer	Reviews aggregate anonymised analytics
Governance	CDSCO Regulatory	Periodic audit of safety engine

14. Data Sources & Knowledge Base

14.1 Disease Dataset

- **146 diseases** across 7 priority tiers (A–G)
- Source: `DhanwantariAI_Symptom_Disease_Mapping.json`
- Each disease includes: symptoms, confirmation tests, generic medicines, JanAushadhi alternatives, Ayurvedic remedies, India-specific notes, gender tag, category tag

14.2 Clinical Data Sources

Category	Sources
Clinical Guidelines	WHO IMCI 2014, ICMR Guidelines, RNTCP 2019, NVBDCP 2021, NPCB, FOGSI MNHRC 2019, NHM STGs
Medicine Databases	NLEM 2022, JanAushadhi (PMBJP) catalogue, AYUSH formulary
Disease/Symptom Datasets	DhanwantariAI curated 146-disease JSON, MedMCQA, DiseaseSymptomKB, ICD-10 codes
Government Programmes	NHM protocols, RBSK screening, Ayushman Bharat, PMJAY
Standards	ICD-10 (2019), SNOMED-CT Indian extension, ATC

14.3 PageIndex Trees (Offline)

Bundled JSON trees via `require()` in the JS bundle: - `clinical_protocols_pageindex.json` — disease management protocols - `referral_guidelines_pageindex.json` — NHM-aligned referral pathways

15. Multilingual Architecture

Language Pack System (from model_variants.json)

Language	Code	Status	Notes
English	en	Released	Base training language
Hindi	hi	Planned	Priority #1; strong Gemma 3 coverage
Odia	or	Planned	Priority #2; native speaker curation; IndicCorp v1 107M tokens
Marathi	mr	Future	—
Kannada	kn	Future	—
Tamil	ta	Future	—
Telugu	te	Future	—

Architecture

```
Base Model (English) — downloaded once
  └── Language Adapter (LoRA, ~40 MB each)
      └── UI Translation Sidecar (JSON, ~3 MB)
          └── System Prompt (training/lang/<code>/system_prompt.txt)
```

Training data generation: `python3 scripts/generate_finetune_dataset.py --language <code>`

---

16. Observability & Telemetry

CloudWatch Metrics (emitted by Lambda)

Metric	Unit	Dimension
`EscalationRate`	Count	DhanwantariAI
`BedrockCost`	None (USD)	DhanwantariAI
`BedrockLatencyMs`	Milliseconds	DhanwantariAI
`BedrockInvokeError`	Count	DhanwantariAI

CloudWatch Alarms → SNS (`DhanwantariAlerts`)

- HighEscalationRate (>40% over 1h)

- BedrockCostSpike (>$3 over 7d)
- BedrockErrors (>10 over 1h)

Client-Side Audit

- Local-only AuditLogger (max 500 entries, auto-rotation)
- No PII in any log entry
- Structured metadata: tier, risk level, confidence, escalation target

---

17. Project File Structure

```
DhanwantariDataset/
├── aws/
│   ├── setup_aws.py                    # AWS infrastructure provisioner
│   ├── aws_outputs.json                # Provisioned ARNs/URLs
│   ├── ingest_vectors.py               # Qdrant vector seeding
│   ├── qdrant_create_collections.py    # Qdrant collection setup
│   └── lambda/bedrock_proxy/index.py   # Lambda handler

├── DhanwantariAI/
│   ├── DhanwantariAI_Symptom_Disease_Mapping.json   # Master 146-disease
dataset
│   ├── DhanwantariAI_Disease_Spreadsheet_Prompt.md  # Dataset generation
prompt
│   ├── scripts/
│   │   ├── fix_truncation.py                          # JSON repair
│   │   └── generate_finetune_dataset.py              # Training JSONL
generator
│   ├── sync/
│   │   ├── bedrock_sync_agent.py      # Agent 1: daily delta + CDN publish
│   │   ├── bedrock_inference_agent.py # Agent 2: Tier 1 LLM inference
│   │   ├── bedrock_kb_agent.py        # Agent 3: RAG knowledge base
│   │   ├── client_sync_engine.py      # Reference sync policy engine
│   │   ├── model_variants.json        # Single source of truth for models
│   │   └── sync_manifest_schema.json  # Manifest JSON schema
│   ├── training/
│   │   ├── train.py                   # Unified QLoRA fine-tuning script
│   │   ├── lang/{en,hi,or}/           # Language-specific system prompts
│   │   └── output/                    # Trained model outputs
│   └── training_data/
│       ├── dhanwantari_train.jsonl     # English training data
│       ├── dhanwantari_val.jsonl       # English validation data
│       └── lang/hi/                   # Hindi training data

├── DhanwantariMobile/
│   └── src/
│       ├── config.ts                   # All endpoints, keys, thresholds
│       ├── ai/
│       │   ├── ClinicalSafetyEngine.ts # 18 red-flag rules (RF001–RF018)
│       │   ├── DeviceCapabilityDetector.ts # 5-signal tier detection
│       │   ├── DiagnosisEngine.ts      # Central orchestrator
│       │   └── RuleEngine.ts           # Deterministic risk classification
│       ├── cloud/
│       │   ├── LLMEngine.ts            # 4-tier LLM routing
│       │   ├── BedrockEscalationHandler.ts # TLS-pinned Bedrock proxy call
│       │   └── PIIStripper.ts          # 6-regex PII removal
│       ├── confidence/
│       │   ├── ConfidenceScorer.ts     # 6-signal weighted scorer
│       │   └── ResultReconciler.ts     # 3-way agreement detection
│       ├── retrieval/
│       │   ├── HybridRetrieval.ts      # Parallel FTS+PI+Vec orchestrator
│       │   ├── FTS5Search.ts           # SQLite FTS5 wrapper
│       │   ├── VectorSearch.ts         # Qdrant Cloud REST client
│       │   └── PageIndexNavigator.ts   # Bundled JSON tree search
│       ├── liability/
│       │   ├── SourceCitationEngine.ts # Citation generation
```

```
│       │   ├── DisclaimerManager.ts    # Centralized disclaimer strings
│       │   └── AuditLogger.ts          # No-PII audit trail
│       ├── privacy/
│       │   └── ConsentManager.ts       # DPDP consent management
│       ├── services/
│       │   ├── db.ts                   # op-sqlite + FTS5 + sqlite-vec
│       │   ├── llmApi.ts              # RTK Query for llama.cpp
│       │   ├── offlineFallback.ts     # Pattern-matched offline responses
│       │   ├── reportIndexer.ts       # react-native-pageindex integration
│       │   └── syncEngine.ts          # TypeScript sync policy engine
│       ├── store/
│       │   ├── store.ts               # Redux store + persist config
│       │   ├── types.ts               # All TypeScript type definitions
│       │   ├── profileSlice.ts        # Profile state management
│       │   ├── chatSlice.ts           # Chat session state
│       │   ├── symptomSlice.ts        # Symptom selection state
│       │   └── deviceSlice.ts         # Device tier state
│       └── features/
│           ├── chat/                  # Chat screen
│           ├── symptoms/              # Symptom checker
│           ├── classifications/       # Disease classification
│           ├── medicine/              # Medicine search
│           ├── referral/              # Referral guidance
│           ├── consent/               # Consent collection
│           └── profile/               # Profile management

└── Doc/
    ├── DhanwantariAI_Architecture.md           # v2.1
    ├── DhanwantariAI_Architecture_v2.2.md      # v2.2 (production-grade)
    ├── DhanwantariAI_Architecture_TODO_DataSources.md
    ├── DhanwantariAI_TODO_Progress.md
    ├── DhanwantariAI_TODO_AWS.md
    └── DhanwantariAI_TODO_Device_Mobile.md
```

---

18. Implementation Status

Completed (✅)

Datasets & Training: - 146-disease symptom mapping JSON (7 priority tiers) - Training JSONL + validation JSONL (ShareGPT format) - QLoRA fine-tuning pipeline (train.py + model_variants.json) - Trained output: `gemma3-1b-dhanwantari-ft-Q4_K_M.gguf`

AWS Infrastructure: - Lambda proxy (DhanwantariBedrockProxy) ✅ - API Gateway (POST /escalate, 50 RPS) ✅ - DynamoDB rate limiting ✅ - Bedrock Guardrails ✅ - CloudWatch alarms + SNS alerts ✅ - Qdrant Cloud collections ✅

Mobile — Safety (P0): - ClinicalSafetyEngine (18 rules) ✅ - RuleEngine (IMMEDIATE/URGENT/ROUTINE) ✅

Mobile — Confidence (P1): - ConfidenceScorer (6-signal) ✅ - ResultReconciler ✅

Mobile — Liability (P2): - SourceCitationEngine ✅ - DisclaimerManager ✅ - AuditLogger ✅

Mobile — Core Infrastructure: - DeviceCapabilityDetector (5-signal) ✅ - DiagnosisEngine (orchestrator) ✅ - LLMEngine (4-tier routing) ✅ - BedrockEscalationHandler (TLS pinned) ✅ - PIIStripper (6 patterns) ✅ - HybridRetrieval (parallel FTS+PI+Vec) ✅ - FTS5Search, VectorSearch, PageIndexNavigator ✅ - ConsentManager ✅ - db.ts (op-sqlite schema) ✅ - syncEngine.ts ✅ - offlineFallback.ts ✅ - Redux store (profile, chat, symptom, device slices) ✅

Sync Agents: - client_sync_engine.py (Python reference) ✅ - bedrock_sync_agent.py (Agent 1) ✅ - bedrock_inference_agent.py (Agent 2) ✅ - bedrock_kb_agent.py (Agent 3) ✅ - model_variants.json ✅

Pending / In Progress

- On-device LLM integration (llama.rn bindings) — Tier 1 in LLMEngine is a stub
- TFLite embedding model (MiniLM-L6-v2) — current `embedText()` returns zero vector
- Language packs (Hindi adapter training, Odia dataset curation)
- Bedrock Knowledge Base creation (Agent 3 deployment)
- KB patch architecture (OTA delta for clinical protocols)
- Bias & safety evaluation framework (200-vignette benchmark)
- Clinical validation framework (4 stages)

End of Document